
📝 Engineering Log: RPLIDAR S3 Integration & Troubleshooting on ROS 2 Humble

This document chronicles the step-by-step setup, optimization adjustments, and troubleshooting solutions discovered while interfacing a Slamtec RPLIDAR S3 with a Raspberry Pi and streaming data to a remote laptop.

🏗️ Phase 1: Workspace Initialization & Compilation Optimizations

1. Source Tree Setup & Repository Cloning

The repository was targeted to build the essential laser driver nodes within a designated ROS 2 workspace workspace.

```
``bash
```

```
# Check if the Pi natively detects the USB serial converter hardware link
ls /dev/ttyUSB0
```

```
# Initialize the workspace source directory
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

```
# Clone the targeted ROS2 branch of the Slamtec RPLIDAR driver repository
git clone -b ros2 https://github.com/Slamtec/rplidar_ros.git
```

```
...
```

2. Low-Memory Resource Optimizations (The Pi Freezing Issue)

> 🛑 ****Problem Encountered:**** The default multi-threaded compilation command (`colcon build`) caused the Raspberry Pi to freeze or hang indefinitely during the cloning and build process. This occurred because `colcon` attempted to compile multiple tasks concurrently, exhausting the Pi's available system RAM.

> 🛠️ ****Discovered Solution:**** Cleaned the broken build environment data and forced a sequential, resource-constrained compilation profile. This ensures the Pi compiles tasks one by one to avoid heavy disk swapping.

```
``bash
```

```
cd ~/ros2_ws
# Wipe out previous corrupted build artifacts safely
rm -rf build/ install/ log/
```

```
# Force sequential single-core build execution parameters
colcon build --executor sequential --parallel-workers 1 --packages-select rplidar_ros
```

...

🌐 Phase 2: Solving the Network Firewall Deficit (Why standard topics didn't connect)

The Networking Problem

Standard ROS 2 network communications (`ros2 topic list`) completely failed to establish a handshake between the Pi and the Laptop's native WSL2 system. This was caused by strict network boundary isolation and local Windows Firewall rules blocking inbound discovery traffic.

The Network Solution Path

Two separate network streaming architectures were tested before locking down a highly reliable data pipeline:

Attempt A: The Rosbridge WebSocket Protocol (Legacy Setup)

An execution pathway to expose the ROS 2 graph to a web-friendly JSON format using port-forwarding over port `9090`.

1. Installed the web bridge interface on the Pi:

```
``bash
sudo apt update && sudo apt install -y ros-humble-rosbridge-server
```

...

2. Launched the background streaming socket server on the Pi:

```
``bash
source /opt/ros/humble/setup.bash
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

...

3. Established a direct local loop tunnel on the Laptop's local WSL2 terminal:

```
``bash
ssh -L 9090:localhost:9090 newpi@10.184.1.113
```

...

4. Connected Windows Foxglove Studio via the `ROS 2 Rosbridge` interface targeting: `ws://localhost:9090`.

Attempt B: The Native Foxglove WebSocket Protocol (Final Production Setup)

>  ****Optimization Note:**** Rosbridge was updated to the native `foxglove\_bridge` setup. This provides much cleaner data packet packing, natively matching the internal serialization format used by modern Foxglove visualizer clients.

🚀 Phase 3: Complete Step-by-Step Clean Startup Sequence (From Scratch)

When initializing the entire robot assembly stack from a dead stop, execute these steps in exact order while sharing a single mobile hotspot connection.

Step 1: Initialize Lidar Driver Component (Pi Terminal Tab 1)

This node boots up the physical sensor hardware over the raw system USB serial interface.

```
``bash
source /opt/ros/humble/setup.bash
# Grant immediate read/write communication clearance to the raw USB bus link
sudo chmod 666 /dev/ttyUSB0
# Launch the specific S3 sensor parameters package
ros2 launch rplidar_ros rplidar_s3_launch.py
```

...

Verification Check: Confirm the logs terminal prints out `RPLidar health status : OK` and `current scan mode: DenseBoost` lines.

Step 2: Establish the Native Network Data Bridge (Pi Terminal Tab 2)

This node starts the stream protocol that converts live internal binary topics into external network packets.

```
``bash
source /opt/ros/humble/setup.bash
# Fire up the bridge node targeting network port 8765
ros2 launch foxglove_bridge foxglove_bridge_launch.xml port:=8765
```

...

Verification Check: Confirm the screen logs output `Advertising new channel` specifically targeting the `/scan` topic.

Step 3: Penetrate the Windows Firewall via SSH Loop Tunnel (Laptop WSL2 Terminal)

Run this command locally on your laptop to link your machine's port network architecture straight inside the Pi's system network.

```
```bash
ssh -L 8765:localhost:8765 newpi@10.184.1.113
```

```

*Verification Check: Authenticate via your password link, and **leave this terminal open** to keep the encrypted path alive.*

Step 4: Interface Configuration Layout within Foxglove Studio

1. Launch **Foxglove Studio** directly on your Windows desktop interface as a standard application.
2. Hit **Open a live connection** and choose **Foxglove WebSocket** from the selection list.
3. Paste the loop network path: ``ws://localhost:8765`` and click **Connect**.
4. Inside the 3D panel settings sidebar, manually configure the structural coordinate links to override missing dropdown menus:
 - * **Fixed frame:** Type ``laser`` and hit Enter.
 - * **Display frame:** Type ``laser`` and hit Enter.
5. In the topic selection tree list, find the active ``/scan`` topic channel and click the open **Eye icon** to start rendering the live laser tracking arrays.

⚠ Crucial Troubleshooting Notes to Remember

- * **Dropdown Menus Empty/No Options:** This always indicates that the main ``rplidar_node`` on the Pi isn't running or stopped halfway. Ensure Step 1 didn't crash out with a USB disconnect.
- * **Typing the Frame Name:** If ``laser`` isn't showing up as a choice in Foxglove's menus, do not worry—manually type out ``laser`` directly inside the text configuration box and hit **Enter** to bind the coordinate frames.